

Herald



Herald Constitution

Field	Value
Revision	7
Created	2026-05-15
Last modified	2026-05-27
Status	active
Status summary	r7: extended §108 with §108.j–§108.l restating the next three inherited HelixConstitution mandates — §11.4.95 workable-items SQLite DB tracked-in-git-not-gitignored, §11.4.96 safe-parallel-work-with-long-build catalogue + mandate, §11.4.97 maximum-use-of-idle-time + progress-update cadence — at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined; literal anchors 11 . 4 . 95/11 . 4 . 96/11 . 4 . 97 now present in all three Herald root docs. Prior r6: extended §108 with §108.d–§108.i restating the next six inherited HelixConstitution mandates — §11.4.89 background-test execution, §11.4.90 Obsolete status + obsolescence audit, §11.4.91 summary-doc clarity, §11.4.92 multi-pass change-evaluation, §11.4.93 SQLite workable-items SSoT, §11.4.94 zero-idle parallel-by-default — at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined; literal anchors 11 . 4 . 89–11 . 4 . 94 now present. Prior r5: added §108 (with §108.a/b/c) restating the three inherited HelixConstitution mandates — §11.4.85 stress + chaos test mandate, §11.4.87 endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing, §11.4.88 background-push — at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined. Required by the §11.4.87 CM-COVENANT-114-87-PROPAGATION pre-build gate. Prior r4: added §107 End-user-usability covenant (verbatim operator mandate restated at Herald level per §1.1 propagation) + paired gate invariant I8a–c; corrected stale "Owned-submodule set: (none)" to reflect the 10 vendored modules (9 Helix-stack + containers).
Issues	none
Issues summary	—
Fixed	R-14 (V2), V3-path-sync (V3 r3), §107 mandate + I8 gate invariant + owned-submodule list (r4), §108 Helix §11.4.85 + §11.4.87 + §11.4.88 propagation (r5), §108.d–§108.i Helix §11.4.89–§11.4.94 propagation (r6), §108.j–§108.l Helix §11.4.95–§11.4.97 propagation (r7)

Field	Value
Fixed summary	§106 spec-change rule retargeted to V3 path; gate-checked anchor 'comprehensive planning and implementation' unchanged so I7 stays green. r4 adds the end-user-usability covenant at Herald level (Helix §11.4 is the canonical authority; Herald restates per §1.1 multi-file propagation discipline) and the I8 gate invariant that asserts the mandate is present in CLAUDE.md + AGENTS.md + this file. r5 adds §108 (§108.a stress+chaos §11.4.85, §108.b endless-loop autonomous work §11.4.87, §108.c background-push §11.4.88) restating the three new inherited HelixConstitution mandates at Herald level per the §11.4.87 CM-COVENANT - 114 - 87 - PROPAGATION pre-build gate, inherited per §11.4.35; restated + cited, not redefined. r6 extends §108 with §108.d–§108.i (§108.d background-test execution §11.4.89, §108.e Obsolete status + obsolescence audit §11.4.90, §108.f summary-doc clarity §11.4.91, §108.g multi-pass change-evaluation §11.4.92, §108.h SQLite workable-items SSoT §11.4.93, §108.i zero-idle parallel-by-default §11.4.94) restating the next six inherited HelixConstitution mandates at Herald level, inherited per §11.4.35; restated + cited, not redefined; literal anchors 11.4.89/11.4.90/11.4.91/11.4.92/11.4.93/11.4.94 now present in all three Herald root docs. r7 extends §108 with §108.j–§108.l (§108.j workable-items SQLite DB tracked-in-git-not-gitignored §11.4.95, §108.k safe-parallel-work-with-long-build catalogue + mandate §11.4.96, §108.l maximum-use-of-idle-time + progress-update cadence §11.4.97) restating the next three inherited HelixConstitution mandates at Herald level, inherited per §11.4.35; restated + cited, not redefined; literal anchors 11.4.95/11.4.96/11.4.97 now present in all three Herald root docs.

Continuation —

Table of contents

- [Project Articles](#)
 - [§101. Pre-implementation status](#)
 - [§102. Mission boundary](#)
 - [§103. Mirror parity \(extends Universal §2.1\)](#)
 - [§104. No embedded constitution \(extends Universal §3\)](#)
 - [§105. Inheritance gate \(extends Universal §1.1\)](#)
 - [§106. Spec-change rule \(extends Universal §11.4\)](#)
 - [§107. End-user-usability covenant \(extends Universal §11.4 — MANDATORY ANTI-BLUFF\)](#)
 - [§108. Inherited covenant restatements \(Helix §11.4.85 / §11.4.87 / §11.4.88 / §11.4.89 / §11.4.90 / §11.4.91 / §11.4.92 / §11.4.93 / §11.4.94 / §11.4.95 / §11.4.96 / §11.4.97\)](#)
- [Overrides of Universal Constitution](#)
- [Owned-submodule set \(per Universal §4\)](#)
- [Project-specific remotes](#)
- [Notes](#)

This constitution **extends** the Helix Universal Constitution provided by the **parent project's** `constitution/ submodule`. Herald does not carry its own copy — see `docs/guides/CONSTITUTION_INHERITANCE.md` for the discovery contract.

All clauses in the parent-provided `constitution/Constitution.md` apply unless explicitly overridden below with an explicit `Override §X.Y` section.

Canonical constitution repo: <https://github.com/HelixDevelopment/HelixConstitution>

Project Articles

§101. Pre-implementation status

Herald is pre-implementation. Until a `go.mod` is committed, no clause below may be interpreted as authorizing the agent to fabricate build/test infrastructure that doesn't yet exist. Confirm the disambiguation (scaffold vs. fill spec) with the operator before writing code, per Universal §11.4.6 (no-guessing).

§102. Mission boundary

Herald ingests system events and fans them out to multiple notification channels. Anything outside event ingestion or notification fan-out is **out of scope** for this repo and belongs in a different submodule of the consuming project.

§103. Mirror parity (extends Universal §2.1)

Every commit on `main` MUST land on all four upstream hosts (GitHub, GitLab, GitFlic, GitVerse) in a single fan-out push. The repo's `origin` remote already aggregates the four push URLs; do not bypass `origin` with per-host pushes unless rebuilding the fan-out configuration.

§104. No embedded constitution (extends Universal §3)

Herald **MUST NOT** carry its own `constitution/` submodule. Herald is consumed as a submodule of a parent project that already provides the constitution; a duplicated copy would diverge in pinning, cause confusion about which is authoritative, and violate the "submodule commits propagate first" propagation order (Universal §3). The inheritance gate's invariant I6 enforces this: it FAILs if `<repo-root>/constitution/` or `.gitmodules` reappears.

For standalone development of Herald (no parent project), clone the constitution **alongside** Herald, not inside it:

```
git clone git@github.com:HelixDevelopment/HelixConstitution.git \
    $(dirname "$PWD")/constitution
```

§105. Inheritance gate (extends Universal §1.1)

`tests/test_constitution_inheritance.sh` discovers the parent-provided constitution via inline parent-walk (mirrors `find_constitution.sh` Phase 1), then asserts six invariants:

Invariant

- I1 A `constitution/Constitution.md` is reachable by walking up the parent chain from Herald's repo root.
- I2 The discovered `Constitution.md` contains the §11.4 End-user quality guarantee — forensic anchor line (the exact string the §1.1 mutation removes).
- I3 The discovered `CLAUDE.md` contains the MANDATORY ANTI-BLUFF COVENANT anchor.
- I4 The discovered `AGENTS.md` contains the Anti-bluff covenant anchor.
- I5a–d Herald's root docs (`CLAUDE.md`, `AGENTS.md`, this file, and `README.md`) all declare the parent-discovery inheritance contract.

Invariant

- I6 No constitution/ directory or .gitmodules file exists at Herald's root (the §104 invariant).
- I7a–c Herald's CLAUDE .md, AGENTS .md, and this file all contain the §106 spec-change rule anchor (per §1.1 propagation, mutation-paired).
- I8a–c Herald's CLAUDE .md, AGENTS .md, and this file all contain the §107 end-user-usability covenant anchor (the verbatim operator-mandate quote — per §1.1 propagation, mutation-paired).

tests/test_constitution_inheritance_meta.sh delegates to the discovered constitution's meta_test_inheritance.sh, which strips the §11.4 anchor from Constitution.md, runs the gate, and asserts FAIL — proving the gate is not a bluff (Universal §1.1).

Both scripts run as a precondition to any commit that touches root docs or the discovery contract.

§106. Spec-change rule (extends Universal §11.4)

Any modification to docs/specs/mvp/specification.V3.md or any file under docs/specs/ (any depth) triggers **mandatory comprehensive planning and implementation of all changes**. An agent or contributor may not edit the spec in isolation: every change is a project-wide ripple that requires the corresponding code, tests, and downstream doc updates in the same logical work effort.

This rule does NOT apply to creating or renaming files; for those, the operator must explicitly tell the worker (CLI agent or human contributor) what to do with the newly created or copied files.

Propagation. The same rule is restated in Herald's CLAUDE .md and AGENTS .md (per §1.1 multi-file propagation discipline). The inheritance gate's invariant **I7a–c** asserts the rule's anchor literal (comprehensive planning and implementation) is present in all three files; a missing copy is a propagation bluff and the gate FAILs.

Anchor (forensic): the literal text Whenever this document (\docs/specs/mvp/specification.V3.md')MUST appear indocs/specs/mvp/specification.V3.md' §"Specification documents" — that line is the source of truth that all three propagated copies summarize.

Paired §1.1 mutation (planned). Removing the spec-change anchor from any of the three propagation files MUST cause I7a/b/c to FAIL; the paired meta-test will be added when test_constitution_inheritance_meta.sh is generalised beyond its current single-anchor mutation.

§107. End-user-usability covenant (extends Universal §11.4 — MANDATORY ANTI-BLUFF)

Forensic anchor — verbatim operator mandate (first declared 2026-04-28, reasserted 2026-05-19 and 2026-05-20):

"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our

project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

Canonical authority. Helix Universal Constitution §11.4 + §11.4.1..§11.4.16, restated at the parent's `constitution/CLAUDE.md` "MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE" section and at the parent's `constitution/AGENTS.md` matching section. Herald inherits the covenant unconditionally; §107 exists to make the restatement explicit at Herald level and to bind every Herald binary (`pherald`, `sherald`, `cherald`, `bherald`, `rherald`, `iherald`, `scherald`, ...) to the covenant on its own terms.

Operative rule (Herald-binding).

1. The bar for shipping any Herald feature is **NOT** "tests pass" or "the binary compiles" — it is **"the end user of <flavor>herald can actually use the feature."**
2. Every PASS — unit test, integration test, gate, Challenge, smoke test, e2e — MUST carry positive runtime evidence that the user-visible behaviour works. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-only PASS without runtime evidence are §11.4 PASS-bluffs and constitute critical defects regardless of how green the summary line looks.
3. Tests AND Challenges are bound **equally**. A Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.
4. The canonical Herald enforcement is `scripts/e2e_bluff_hunt.sh` — it builds `pherald`, runs the full test suite, starts a real Gin server, hits every `/v1/*` route, asserts response bodies, boots a real Postgres container via the `containers/` submodule, runs the M2 integration tests against the live DB, and SIGTERM-graceful-shutdowns. A single FAIL invariant means a real feature is broken for end users; no release tag, no risky commit, and no "implementation milestone landed" claim may be made while it FAILs.
5. New user-visible Herald features (V3 §§11, 33, 41, 42, 43 and beyond) MUST extend `e2e_bluff_hunt.sh` with a new `E_N` invariant in the same logical work effort — a feature that ships without its e2e invariant is shipping without anti-bluff evidence and violates §107.

Propagation. This §107 is restated verbatim (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md` and `AGENTS.md` per §1.1 multi-file propagation discipline. The inheritance gate's invariant `I8a-c` asserts the anchor literal (`End-user-usability covenant` or the verbatim "all tests do execute with success" operator quote) is present in all three files; a missing copy is a propagation bluff and the gate FAILs.

Paired §1.1 mutation. A future generalised mutation-meta will assert that removing the §107 anchor from any of the three propagation files MUST cause `I8a/b/c` to FAIL — the §1.1 paired-mutation discipline is non-negotiable for every new gate.

Non-compliance is a release blocker. No flavor binary may be tagged, no submodule may be propagated, and no operator-handoff (`docs/CONTINUATION.md`) may be published while a §107 evidence-gap is open.

§107.x. docs/qa/ Evidence Mandate (operator mandate, 2026-05-22 — extends §107; cascades from Helix §11.4.83)

Forensic anchor — verbatim operator mandate (2026-05-22):

"every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A

feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation (e.g. Herald's planned qaherald binary) MUST preserve full bidirectional communication threads as proof."

Canonical authority. Helix Universal Constitution §11.4.83 (the rule was added at universal level in the same propagation cycle as this §107.x). Herald §107.x is the project-binding restatement.

Operative rule (Herald-binding).

1. Every Herald feature that ships — every flavor binary route (`/v1/events`, `/v1/compliance`, `/v1/safety_state`, ...), every Telegram bot interaction (HRD-011), every Claude Code dispatch path (HRD-012), every container-orchestrated flow, every QA-bot transcript (planned qaherald per HRD-NNN-to-be-assigned) — MUST carry a `docs/qa/<run-id>/` directory committed in the same logical work effort (per V3 §8.3 HRD lifecycle). `<run-id>` is monotonic + greppable: HRD-NNN, HRD-NNN-`<TS>` (multi-run), or a free `<TS>` tag.
2. Transcripts are **full bidirectional** — for Telegram: both user→bot and bot→user halves; for Gin: both request payload (JSON or TOON per Wave 4b) and full response body + status line + relevant headers; for Claude Code: both the prompt sent and the response received; for container flows: container stdin → container stdout/stderr + container logs.
3. Attached materials are committed **in-repo**, never linked. Screenshots in `.png`; JSON / TOON payloads as their natural extension; container logs as `.log`; OpenTelemetry trace exports as `.json` / `.otlp`. External-only links (Slack URL, Drive URL, Telegram message URL) are §11.4.13 sink-side violations — the artefact lives in `docs/qa/<run-id>/`.
4. The planned qaherald binary (HRD-NNN to be assigned) is Herald's authoritative QA automation. It drives pherald [?] Telegram (and analogous round-trips for the other flavors) and preserves the full conversation thread under `docs/qa/qaherald-<TS>`. A qaherald run that stores only the final PASS/FAIL line is itself a §107.x bluff at the QA-automation layer.
5. New `e2e_bluff_hunt` invariants for features added after 2026-05-22 MUST cite their `docs/qa/<run-id>/` artefact as positive-evidence anchor (composes with §11.4.2 / §11.4.5). A new `E_N` invariant without a corresponding `docs/qa/` directory is a §107.x violation in the same logical work effort.
6. Release-gate enforcement: `scripts/release.sh` (when implemented) + the existing tag-time guard MUST refuse to tag a Herald release whose feature-shipping commits since the previous tag lack their matching `docs/qa/<run-id>/` directories.

Propagation. §107.x is restated (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.83 is cascaded into the 11 Helix-stack submodules (`auth`, `background`, `cache`, `Concurrency`, `config`, `database`, `eventbus`, `middleware`, `Models`, `observability`, `recovery`) per §1.1 multi-file propagation discipline.

Non-compliance is a release blocker. No `--qa-evidence-optional`, `--qa-transcript-later`, `--qa-bot-summary-suffices` flag exists.

§107.y. Working-Tree Quiescence Rule (operator mandate, 2026-05-22 — extends §107; cascades from Helix §11.4.84)

Short tag: working-tree quiescence.

Forensic anchor — verbatim operator mandate (2026-05-22):

"no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired, // always pass, return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."

Canonical authority. Helix Universal Constitution §11.4.84. Herald §107.y is the project-binding restatement.

Lesson (forensic case study — Herald-internal, 2026-05-21). A logo-fix subagent (commit 72e81ab, "Fix: replace pandoc {width=96px} image attr with HTML tag") ran in this very checkout while a paired §1.1 Wave 4b mutation gate had temporarily injected an `// always pass` shortcut into `commons_auth/middleware.go` (JWT-bypass mutation, intended for the mutate → assert FAIL → restore cycle). The subagent's `git add` swept the mutation residue into its commit; the commit was pushed to all four mirrors before any other agent caught it. Within the hour the SECURITY FIX (d5bd360, "SECURITY FIX: restore commons_auth/middleware.go JWT verify (mutation residue in 72e81ab)") restored the verify path. But the production-equivalent-binary-with-bypassed-JWT window is a real security-defect window — small, but non-zero, and demonstrably exploitable in that interval. The rule below is the constitutional outcome. This is no longer a hypothetical; it is documented Herald history.

Operative rule (Herald-binding).

1. **Pre-git add quiescence check.** Every commit flow (main thread + every dispatched subagent) MUST `grep` the working tree for the canonical Herald mutation markers BEFORE `git add`:
 - `MUTATED for paired` (the canonical paired-§1.1 marker emitted by `tests/test_wave4b_mutation_meta.sh` + future generalisations)
 - `// always pass, // MUTATION, # MUTATION` (Go + shell mutation annotations)
 - `return json.Marshal` shortcut paths in `commons/` or `commons_messaging/` (Wave 4b TOON mutation residue)
 - `_mutated_*` filename suffixes
 - `.git/MUTATION_IN_PROGRESS` (the lockfile)
2. **Scope-match.** Cross-check `git status --porcelain` against the subagent's declared scope. Any file outside the declared scope → ABORT. The subagent MUST explicitly account for every modified / untracked / staged entry.
3. **Lockfile serialisation.** When any mutation gate is in flight, its first action is `touch .git/MUTATION_IN_PROGRESS`; its last action (trap-on-exit) is `rm .git/MUTATION_IN_PROGRESS`. Any subagent finding this lockfile present MUST refuse to `git add` and ABORT until the gate completes its mutate → assert FAIL → restore → assert PASS cycle and removes the lockfile.
4. **Worktree isolation (preferred).** When parallel subagents are required (§11.4.20 / §11.4.70 subagent-driven default), prefer `git worktree add` per subagent over single-checkout concurrency — eliminates the cross-mutation race by construction.
5. **Pre-push mutation-residue scanner.** `scripts/mutation_residue_audit.sh` (planned, HRD-NNN to be assigned) MUST run before every push. Any commit in the pushed range containing a mutation marker → push BLOCKED.

Prototype. `tests/test_wave4b_mutation_meta.sh` ALREADY includes the canonical Herald implementation:

- `check_quiescence()` at line 92 — the working-tree quiescence guard (returns 0 if NO MUTATED markers in tracked files).
- Line 197 — the "Working-tree quiescence — assert no MUTATED markers leaked" assertion.

The Wave 4b test is the prototype; generalising the check across every paired-§1.1 gate (Wave 2, Wave 3, Wave 4a) is open work. The planned universal scanner `scripts/mutation_residue_audit.sh` is the roll-out vehicle.

Composes with §107 (a security-bypass mutation that ships to production is the gravest §107 PASS-bluff), §1.1 (paired-mutation discipline — the rule protects the mutation cycle from concurrent contamination), §11.4.20 / §11.4.70 (subagent-driven default — quiescence rule makes parallel subagent dispatch safe), §11.4.10 (credentials handling — same class of "no unrelated content in a commit"), §11.4.27 (no-fakes-beyond-unit — a mutation residue swept into a commit IS a fake-pass surface in production).

Propagation. §107.y is restated (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.84 is cascaded into the 11 Helix-stack submodules per §1.1 multi-file propagation discipline.

Non-compliance is a release blocker. A mutation marker that lands in a tagged Herald commit is a critical defect regardless of how briefly it persisted — see commits `72e81ab` / `d5bd360` as forensic proof. No `--allow-residue`, `--skip-quiescence`, `--mutation-cleanup-later` flag exists.

§108. Inherited covenant restatements (Helix §11.4.85 / §11.4.87 / §11.4.88 / §11.4.89 / §11.4.90 / §11.4.91 / §11.4.92 / §11.4.93 / §11.4.94 / §11.4.95 / §11.4.96 / §11.4.97)

These twelve mandates are **inherited** from the Helix Universal Constitution via parent-discovery (§11.4.35). Herald **restates + cites** them at project-constitution level per the §1.1 multi-file propagation discipline — it does **NOT** redefine, narrow, or weaken them; the parent `constitution/Constitution.md` is the canonical authority for each. The literal anchors (11.4.85, 11.4.87, 11.4.88, 11.4.89, 11.4.90, 11.4.91, 11.4.92, 11.4.93, 11.4.94, 11.4.95, 11.4.96, 11.4.97) are required across Herald's `CLAUDE.md` / `AGENTS.md` / `QWEN.md` by the §11.4.87 CM-COVENANT-114-87-PROPAGATION pre-build gate, which strips the literal in a paired §1.1 meta-test mutation and asserts the gate FAILs.

§108.a. Stress + Chaos Test Mandate (extends Universal §11.4.85; Helix 2026-05-24)

Forensic anchor — verbatim user mandate (2026-05-24):

"Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Canonical authority. Helix Universal Constitution §11.4.85. Herald §108.a is the project-binding restatement.

Operative rule (Herald-binding). Every fix or improvement landed in Herald MUST ship with full-automation **stress** AND **chaos** test suites. Stress = sustained load ($N \geq 100$ iterations or ≥ 30 s, p50/p95/p99 recorded) and/or concurrent contention ($N \geq 10$ parallel, no deadlock / leak / data race) and/or boundary conditions (empty / max / off-by-one, each producing a categorised result). Chaos = at least one failure-injection category appropriate to the fix-class: process-death, network-fault, input-corruption, resource-exhaustion (disk-full / OOM / fd-exhaustion), or state-corruption (DB-lock-loss / partial-write / cache-invalidation). Every stress + chaos PASS cites a captured-evidence artefact under `docs/qa/<run-id>/stress_chaos/` per §11.4.5 + §11.4.69 (`latency.json` / `throughput.csv` / `categorised_errors.txt` / `recovery_trace.log`); chaos-injection cleanup is non-negotiable (`trap '...' EXIT` restores corrupted `.env`, `rms` disk-fillers, verifies killed processes restart). For Herald this binds the flavor binaries: `pherald listen` inbound long-poll under concurrent Telegram updates, `Gin /v1/*` routes under sustained load, `claude_code` dispatch under process-death, container-orchestrated flows under disk/OOM pressure. A happy-path-only PASS is a §107 / §11.4 PASS-bluff at the resilience layer. No `--skip-stress` / `--no-chaos` / `--happy-path-suffices` / `--stress-test-later` escape exists.

§108.b. Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing (extends Universal §11.4.87; Helix 2026-05-26)

Forensic anchor — verbatim user mandate (2026-05-26, condensed):

"all work MUST BE continued in endless loop until there is no any open items, no unfinished workable items from our Issues docs, or from Continuation document or any unfinished work by agents ... fully autonomously. You will spawn agents or agents-driven work whenever that is possible or required! Not a single agent or main work stream will sit idle except if it waits for the results of something ... All work MUST BE always covered with comprehensive tests ... which produce real proofs ... and in complete anti-bluff manner!"

Canonical authority. Helix Universal Constitution §11.4.87. Herald §108.b is the project-binding restatement.

Operative rule (Herald-binding). When the operator instructs Herald work to "continue in endless loop fully autonomously" (or equivalent), it is a HARD-CONTRACT covenant: (A) **continuation** — continue until ALL are simultaneously TRUE: Herald's autonomous loop checks `docs/Issues.md` Status-column has zero In progress / Ready for testing / In testing / Reopened entries (§11.4.15 closed-set), `docs/CONTINUATION.md` §3 "Active work" is empty, the TaskList reports no subagent mid-execution, and no in-flight external dependency (build, push, sync) remains; (B) **zero-idle dispatch** — dispatch background subagents for parallelisable, non-file-scope-contending work rather than serialising; idle is permitted ONLY while waiting on a result; (C) **comprehensive test coverage with real (physical) proofs** — every closed item lands four-layer coverage per §11.4.4(b) with captured-evidence PASS; (D) **anti-bluff end-to-end** — tests AND Challenges are bound equally (a Challenge that scores PASS on a non-functional feature is the same defect class as a unit test that does); (E) **termination** — only on all-clear, an explicit operator STOP / END LOOP, a §12 host-session-safety demand, or a scheduled wake against a known-future-actionable signal. Composes with §107 (anti-bluff) and §12.10 (CONTINUATION-doc maintenance — the source-of-truth state the loop checks). No `--idle-OK` / `--skip-endless-loop` / `--bluff-permitted-for-this-task` / `--metadata-only-test-suffices` / `--no-physical-proof-required` escape exists.

§108.c. Background-push mandate: commit-lock release immediately after commit, push runs detached (extends Universal §11.4.88; Helix 2026-05-26)

Forensic anchor — verbatim user mandate (2026-05-26, condensed):

"Make sure all these pushes are being done ALWAYS in background in parallel with main work stream so we do not loose time waiting. Everything is committed anyway? ... We MUST ensure that main work stream has always something to do, or wait for the results only when that is absolutely required!"

Canonical authority. Helix Universal Constitution §11.4.88. Herald §108.c is the project-binding restatement.

Operative rule (Herald-binding). (A) **Flock release immediately after commit** — once `git commit` returns 0 (commit object durable on local disk), the `.git/.commit_all.lock` flock MUST be released BEFORE any push runs; gating further local work on a remote round-trip is the exact zero-idle anti-pattern §108.b / §11.4.87 prohibits. (B) **Push runs detached** — `nohub ./push_all.sh ... > <log> 2>&1 &` then `disown`; the orchestrator's exit code reports COMMIT success, not push success. (C) **Per-remote serialisation, multi-remote parallelism** — `push_all.sh` acquires a per-remote flock (`.git/.push.<remote>.lock`) so same-remote invocations serialise (no non-fast-forward race) while Herald's four mirrors (GitHub / GitLab / GitFlic / GitVerse) push in parallel. (D) **Failure surface** — backgrounded push failures land in `qa-results/push_failures/<TS><remote>.log`; the next autonomous-loop tick MUST check that directory (the §108.b(A) "no external dependency in-flight" check) and surface failures — silent push-failure is a §11.4 PASS-bluff at the distribution layer. (E) **Synchronous-push escape** — the explicit `--sync-push` flag preserves legacy synchronous behaviour for §11.4.41 force-push merge-first paths; it is the ONLY escape. Composes with §103 (mirror parity — §108.c makes the four-mirror fan-out non-blocking) and §108.b (this is the implementation seam that makes the endless loop genuinely zero-idle for the dominant blocker class).

Propagation. §108 (with §108.a/b/c) is restated (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandates at Helix §11.4.85 / §11.4.87 / §11.4.88 are cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The §11.4.87 CM-COVENANT-114-87-PROPAGATION pre-build gate enforces the literal anchors `11.4.85 / 11.4.87 / 11.4.88` in all three Herald root docs.

Non-compliance is a release blocker. Each sub-clause carries its inherited no-escape-hatch posture: `no --skip-stress / --no-chaos` (§108.a), `no --idle-OK / --skip-endless-loop` (§108.b), `no synchronous push without --sync-push` (§108.c).

§108.d. Background test execution mandate (extends Universal §11.4.89; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"long-running tests (the mutation gates, e2e bluff-hunt, stress and chaos suites) MUST run in background so the main work stream never blocks waiting on them — survey the results from the log when they finish, do not sit idle."

Canonical authority. Helix Universal Constitution §11.4.89. Herald §108.d is the project-binding restatement.

Operative rule (Herald-binding). Any Herald test cycle expected to exceed ~30s — the tests/test_wave*_mutation_meta.sh gates, scripts/e2e_bluff_hunt.sh, future stress/chaos suites — MUST run detached (nohup ... > qa-results/<test_id>_<TS>.log 2>&1 & then disown); the main work stream returns immediately to the §11.4.42 priority queue and polls the log / exit-status rather than blocking. Composes with §107.y / §11.4.84: a backgrounded mutation gate still mutates the shared tree, so only ONE runs against the main checkout at a time (serialised by the .git/MUTATION_IN_PROGRESS lockfile + the planned scripts/mutation_residue_audit.sh pre-push scanner); concurrent gates require separate git worktree checkouts. Foreground is permitted only for <30s tests or on explicit operator request. No --block-on-long-test / --skip-background escape exists.

§108.e. Obsolete status + per-item obsolescence audit (extends Universal §11.4.90; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"some workable items are no longer valid — superseded by a later design or mandate, or the feature was removed. Add an Obsolete status and audit every item for obsolescence at each release, with triple-checked evidence for why it is obsolete — no bare assertions."

Canonical authority. Helix Universal Constitution §11.4.90. Herald §108.e is the project-binding restatement.

Operative rule (Herald-binding). Herald's HRD Status closed-set gains a 4th terminal value Obsolete (→ Fixed.md) for items no longer valid (Reason closed-set: superseded-by-design-change / superseded-by-later-mandate / feature-removed / duplicate-of / unsupported-topology). Every Obsolete HRD heading MUST carry an **Obsolete-Details:** line within 8 non-blank lines of the heading: Since (ISO date), Reason (closed-set value), Superseding-item (§ or HRD reference), and Triple-check evidence (git-log / grep / runtime path per §11.4.6 — a bare assertion is forbidden). At every release-gate sweep, Herald re-evaluates every open + Fixed HRD for obsolescence; Obsolete→Fixed.md migrations are atomic per §11.4.19 (the §107.x docs/qa evidence mandate still applies where the obsoleted item once shipped a feature). No --obsolete-without-evidence / --skip-obsolescence-audit escape exists.

§108.f. Summary-doc clarity (extends Universal §11.4.91; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"the one-line summaries in the Issues/Fixed/Status summary docs are useless when they are just a fragment or a status word — each MUST be a self-contained clause that says what the item is about and what the problem or goal is."

Canonical authority. Helix Universal Constitution §11.4.91. Herald §108.f is the project-binding restatement.

Operative rule (Herald-binding). Every one-liner in docs/Issues_Summary.md / docs/Fixed_Summary.md / docs/Status_Summary.md and every README doc-link row MUST be a self-contained clause (≥6 words OR ≥40 chars) naming the SUBJECT + the PROBLEM/GOAL — never a section-label fragment (Composes with, Closure criteria), bare metadata (Critical, Bug), a status restatement, or a bare HRD-id. Each is derived from the source long-form H1/H2 heading, never invented. The *_Summary.md

variants remain derived (per the documentation-artefacts rule — do not hand-edit them out of sync with source). No `--terse-summary-OK` escape exists.

§108.g. Multi-pass change-evaluation discipline (extends Universal §11.4.92; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"before a change is ready to commit, evaluate it in multiple passes — the main task, the regression blast-radius, the cross-feature interactions, the deep research, and a final anti-bluff confirmation — and document each pass. A change that only proved its happy path is not done."

Canonical authority. Helix Universal Constitution §11.4.92. Herald §108.g is the project-binding restatement.

Operative rule (Herald-binding). Every non-trivial Herald change **MUST** pass — and document — a 5-pass evaluation before it is commit-ready: **Pass 1** main-task captured-evidence (§11.4.5 / §107 — no "should work"); **Pass 2** regression-blast-radius (every touched file + every importer/caller audited — e.g. a `commons` type change audits all flavor binaries); **Pass 3** cross-feature interaction (shared state / timing / env — e.g. a mutation-gate edit checks §107.y quiescence + §108.d / §11.4.89 backgrounding); **Pass 4** deep-research validation (§11.4.8 — external precedent located, or a literal "NO external solution found" recorded); **Pass 5** anti-bluff confirmation (no metadata-only / config-only / script-bug PASS). Evidence lands in the commit footer or under `docs/qa/ / qa-results/`. Trivial changes (typo, revision-bump, MD-export regen touching zero source) are exempt only with an explicit commit-message citation. No `--single-pass-OK` / `--skip-blast-radius` escape exists.

§108.h. SQLite-backed single-source-of-truth for workable items (extends Universal §11.4.93; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"the Issues / Fixed / Status / Continuation tracking is in too many Markdown files that drift out of sync — back it with a single SQLite source of truth and regenerate the Markdown from it (and back) so drift is impossible. The tool already exists in the constitution — use it, do not reinvent it."

Canonical authority. Helix Universal Constitution §11.4.93. Herald §108.h is the project-binding restatement.

Operative rule (Herald-binding). Herald's text-based HRD trackers (`docs/Issues.md` / `docs/Fixed.md` / the `*_Summary.md` variants / `docs/Status.md` / `docs/CONTINUATION.md` §3) migrate to a SQLite single-source-of-truth (`docs/.workable_items.db`, **version-controlled as Herald's authoritative SSoT** — operator mandate 2026-05-27: "We should not git ignore our workable items database since it is our single source of truth regarding items we have to do on the project!"). This is a deliberate Herald divergence from the parent §11.4.93 gitignored-with-regeneration default: for Herald the DB IS the authoritative artefact, so it is committed and version-controlled, NOT regenerated-on-clone; only the transient SQLite WAL/SHM sidecars (`docs/.workable_items.db-wal` / `-shm`) stay gitignored) with bidirectional MD↔DB regeneration so sync-drift is mechanically impossible. The migration Go binary lives in the constitution submodule (`constitution/scripts/workable-items/`) — Herald references it per §11.4.74 catalogue-first, and never reimplements it. The migration is 6-phase; Herald files the tracking HRD (Phase 1) and

progresses incrementally without breaking the §106 spec-change / §107.x evidence invariants. No `--skip-ssot` / `--keep-md-only` escape exists once the migration HRD is filed.

§108.i. Zero-idle priority-first parallel-by-default operating mode (extends Universal §11.4.94; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"never sit idle while there is priority-queued work that could move — survey the queue, pick the highest-priority non-blocking items, and run them in parallel by default. Idle is only allowed when everything is genuinely blocked on something external."

Canonical authority. Helix Universal Constitution §11.4.94. Herald §108.i is the project-binding restatement.

Operative rule (Herald-binding). Binding always-on contract: Herald work is NEVER idle while a priority-queued item can progress. Before any wake / sleep / "waiting for X", survey the priority queue, identify all non-contending items, and dispatch them in parallel — subagent-driven per §11.4.20 / §11.4.70 when non-trivial, backgrounded per §108.d / §11.4.89 when >30s — picking the highest-Severity / priority item first. The conductor remains the integration + commit + push seam; parallel work MUST NOT compromise stability (composes with §107.y quiescence + §108.g / §11.4.92 multi-pass + §12 host-safety). This is the operating-mode generalisation of §108.b / §11.4.87 (endless-loop) and §108.c / §11.4.88 (background-push). Idle is permitted ONLY when every item is genuinely blocked on an external dependency, the operator issued STOP, or §12 host-session-safety demands it. No `--serialise-OK` / `--idle-permitted` escape exists.

Propagation (§108.d–§108.i). These six restatements are summarised — citing this section as the canonical Herald source — in Herald's CLAUDE .md and AGENTS .md (and QWEN .md when present). The universal mandates at Helix §11.4.89 / §11.4.90 / §11.4.91 / §11.4.92 / §11.4.93 / §11.4.94 are cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The literal anchors 11.4.89 / 11.4.90 / 11.4.91 / 11.4.92 / 11.4.93 / 11.4.94 are present in all three Herald root docs.

Non-compliance is a release blocker (§108.d–§108.i). Each sub-clause carries its inherited no-escape-hatch posture: no `--block-on-long-test` (§108.d), no `--obsolete-without-evidence` (§108.e), no `--terse-summary-OK` (§108.f), no `--single-pass-OK` (§108.g), no `--keep-md-only` once migration is filed (§108.h), no `--idle-permitted` (§108.i).

§108.j. Workable-items SQLite DB is TRACKED in git, never gitignored (extends Universal §11.4.95; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"We should not git ignore our workable items database since it is our single source of truth regarding items we have to do on the project! — it must be committed and pushed, never regenerated-on-clone, and never silently rewritten without a backup."

Canonical authority. Helix Universal Constitution §11.4.95. Herald §108.j is the project-binding restatement.

Operative rule (Herald-binding). Herald ALREADY complies (operator correction 2026-05-27, recorded in §108.h + .gitignore): the workable-items SQLite DB is version-controlled, committed + pushed alongside every state change, WAL-checkpointed (PRAGMA wal_checkpoint(TRUNCATE)) before commit so only the transient SQLite -wal/-shm sidecars stay gitignored, and never force-rewritten without a §9.2 hardlinked-backup. This is an explicit §11.4.30 carve-out and AMENDS the earlier §11.4.93 / §108.h "gitignored-with-regeneration" text (which Herald had already diverged from per the same operator mandate). **Herald alignment note:** the constitution's canonical path is docs/workable_items.db (no leading dot); Herald's HRD-131 currently references docs/.workable_items.db — reconcile to the canonical path when the DB is implemented (HRD-131 Phase 2+, currently deferred). No --gitignore-db / --regenerate-on-clone escape exists.

§108.k. Safe-parallel-work-with-long-build catalogue + mandate (extends Universal §11.4.96; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"while a long build or long-running operation is going, do not sit idle — there is a whole catalogue of work that is safe to do in parallel, so consult it and dispatch the safe non-contending items, and only avoid the things that would collide with the running operation."

Canonical authority. Helix Universal Constitution §11.4.96. Herald §108.k is the project-binding restatement.

Operative rule (Herald-binding). Herald has no AOSP build, but the principle binds: during ANY long-running Herald operation (a backgrounded mutation gate, scripts/e2e_bluff_hunt.sh, a §108.a / §11.4.85 stress/chaos suite, a doc export) the conductor MUST consult the safe-parallel catalogue and dispatch non-contending work rather than idle. SAFE-in-parallel for Herald: (A) MD/docs work, (B) scripts/helpers, (C) gate authoring, (D) test authoring, (E/F) commit + push to mirrors, (H) read-only analysis subagents, (I) web research, (J) workable-items DB ops. UNSAFE-during-a-running-gate (maps to §107.y / §11.4.84): git checkout / reset --hard / clean on files a gate is transiently mutating, a SECOND concurrent mutation gate against the same checkout (composes with §108.d / §11.4.89 single-gate serialisation), host-session-safety breaches (§12). Subagent-driven default per §11.4.20 / §11.4.70. This is the catalogue that makes §108.i / §11.4.94 (zero-idle parallel-by-default) concrete during Herald's longest-running operations. No --idle-during-build / --skip-parallel-catalogue escape exists.

§108.l. Maximum-use-of-idle-time mandate + progress-update cadence (extends Universal §11.4.97; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

"use every minute of idle time to move work forward — and keep me posted with short progress updates at each milestone without me having to ask, always backed by real captured evidence for anything you mark done."

Canonical authority. Helix Universal Constitution §11.4.97. Herald §108.l is the project-binding restatement.

Operative rule (Herald-binding). (A) **Maximum-use-of-idle-time** — every minute of conductor idle time during which progressable, non-externally-blocked work exists is a violation; the conductor dispatches work continuously through the whole idle window, not just at scheduled

wakes (the temporal extension of §108.i / §11.4.94 and §108.k / §11.4.96). (B) **Progress-update cadence** — emit concise (1–3 line) operator-facing progress updates at milestone boundaries with NO prompt required: every HEAD advance (what landed), every subagent return (integrated), every constitutional anchor propagated, every captured-evidence artefact (docs/qa/ / qa-results/ path), every Issues→Fixed / Obsolete closure. (C) **Continuous physical-proof gathering** per §11.4.5 / §11.4.6 / §11.4.69 (and §107 / §107.x) — every closed item carries positive captured evidence committed alongside the closure; a "done" with no artefact is a §11.4 / §107 PASS-bluff. (E) Idle is permitted ONLY when every item is genuinely blocked on an external dependency, the operator issued STOP, or §12 host-session-safety demands it. No --idle-permitted / --silent-progress / --evidence-later escape exists.

Propagation (§108.j–§108.l). These three restatements are summarised — citing this section as the canonical Herald source — in Herald's CLAUDE.md and AGENTS.md (and QWEN.md when present). The universal mandates at Helix §11.4.95 / §11.4.96 / §11.4.97 are cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The literal anchors 11.4.95 / 11.4.96 / 11.4.97 are present in all three Herald root docs.

Non-compliance is a release blocker (§108.j–§108.l). Each sub-clause carries its inherited no-escape-hatch posture: no --gitignore-db / --regenerate-on-clone (§108.j), no --idle-during-build / --skip-parallel-catalogue (§108.k), no --idle-permitted / --silent-progress / --evidence-later (§108.l).

Overrides of Universal Constitution

(none — Herald has no exceptions to universal clauses at pre-implementation stage)

Owned-submodule set (per Universal §4)

submodules/auth	→ git@github.com:vasic-digital/auth.git
submodules/background	→ git@github.com:vasic-digital/BackgroundTasks.git
submodules/cache	→ git@github.com:vasic-digital/cache.git
submodules/config	→ git@github.com:vasic-digital/config.git
submodules/database	→ git@github.com:vasic-digital/database.git
submodules/eventbus	→ git@github.com:vasic-digital/EventBus.git
submodules/middleware	→ git@github.com:vasic-digital/middleware.git
submodules/observability	→ git@github.com:vasic-digital/observability.git
submodules/recovery	→ git@github.com:vasic-digital/recovery.git
containers	→ git@github.com:vasic-digital/containers.git

Herald owns 10 vendored submodules — 9 Helix-stack capability modules under submodules/ (referenced via replace directives in consuming Herald modules' go.mod, NOT via go.work) plus containers/ (runtime auto-detection + on-demand container boot, consumed directly by Foundation tests + pherald doctor). Every one of them carries the §11.4 anti-bluff anchor; per Universal §11.4.74 they are catalogue-aware (extend | reuse | no-match discipline). The constitution itself is provided by the parent project, not vendored here — Herald never carries constitution/ (per §104).

Project-specific remotes

Repo	Remotes
Herald (this repo)	github, gitlab, gitflic, gitverse + origin (fan-out push to all four)

Notes

Herald's spec is now in V3 (`docs/specs/mvp/specification.V3.md`, ~3900 lines, active) — V1 and V2 are preserved under `docs/specs/mvp/archive/` for historical reference. As project-specific articles mature toward universal status they may move into the Helix Constitution; promotion requires the §11.4 universal-vs-project audit. Default is to keep rules here until the audit clears.